ERP Electronics Store

Developer & Technical Documentation

Version 2.1 | August 2026

Software Developers · System Administrators · Superadmins

Table of Contents

1. Document Control

Version	2.1
Date	August 2026
Status	Released
Audience	Software developers, system administrators, superadmins, and technical stakeholders
Scope	Every file in both repositories, the full request cycle, architecture, security, deployment, and local-to-production workflow
Related documents	User_Manual_EN.pdf, User_Manual_SW.pdf, Supplier_Manual.pdf, ERD.drawio, ClassDiagram.drawio, UseCase.drawio, SequenceDiagrams.drawio

Repositories

Frontend	`erp-electronics/` — Vue 3 SPA (storefront + dashboards + superadmin panel); this document and all PDF generators live here.
Backend	`erp-electronics-api/` — Laravel 13 REST API.
Documentation	`docs/` in the frontend repository (manuals, diagrams, this document).

Environments

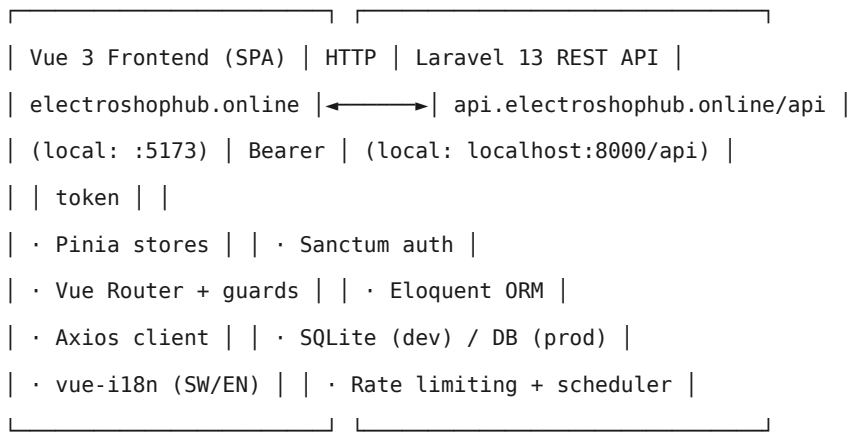
Environment	Frontend	Backend API base
Local development	`http://localhost:5173`	`http://localhost:8000/api`
Production (live)	`https://electroshophub.online`	`https://api.electroshophub.online/api`

The local frontend talks to the local backend. The production build is compiled with VITE_API_URL set to the production API base; the local untracked .env keeps the localhost value, so development and deployment do not interfere.

2. System Overview & Architecture

ERP Electronics Store is a multi-tenant SaaS platform for electronics retail businesses in Tanzania. A platform superadmin registers business owners; each owner runs a white-label online store with their own branding, subscription plan, and resource limits. Owners manage employees, who operate the store — processing orders, managing customers, inventory, and support. Customers shop and pay via mobile money (M-Pesa, Airtel Money, Mixx by Yas, Halopesa, ClickPesa) or cash. Suppliers fulfil purchase orders through a dedicated portal. Winga street promoters drive orders from a per-order commission.

High-Level Architecture



Multi-Tenancy Model

- The tenant key is the owner user id. Every owner-scoped row (products, orders, accounts, commissions, suppliers, wingas, branches, businesses) carries an owner_id foreign key.
- Businesses (multi-store) sit above the owner: a Business row is seeded per owner and can be shared with co-owners via the `business_user` pivot.
- Tenant resolution is centralized in `app/Support/Tenant.php` and exposed as request macros `request()->business()` and `request()->ownerId()`.
- The active business is selected by the `X-Business-Id` request header (sent by the frontend from `active_business_id` in localStorage), falling back to the first owned business.
- Storefront routes scope by a business slug in the URL — e.g. `/:businessSlug` renders that store's branding and catalog.

Request Flow

- The SPA calls the API through an Axios client configured with a base URL (`VITE_API_URL`).
- Authenticated requests send Authorization: Bearer <token> (Sanctum personal access token) and X-Business-Id.
- A 401 response clears the stored token and redirects to the login page.

- API responses are JSON; all `/api/*` routes render JSON errors automatically (`shouldRenderJsonWhen api/*`).
- Public storefront routes (products, categories, businesses, branding, payment providers, shipping estimate) require no authentication.

Cross-Origin Resource Sharing (CORS)

The API permits requests from any origin (allowed_origins: *), which allows both the local Vite server and the production domain to consume the API without CORS configuration changes. Preflight requests are handled automatically.

3. Technology Stack

Frontend

Package	Version	Purpose
vue	^3.5	UI framework (Composition API)
vue-router	^5.1	Client-side routing with role guards
pinia	^3.0	State management (auth, business, cart, products, session)
pinia-plugin-persistedstate	^4.7	Persist the auth token to localStorage
axios	^1.18	HTTP client with auth/business interceptors
vue-i18n	^10.0	Internationalization (Swahili default, English fallback)
@fortawesome/fontawesome-free	^7.3	Icons (fas fa-*)
chart.js + vue-chartjs	^4.5 / ^5.3	Analytics and sales charts
vite	^8.0	Build tooling and dev server
vitest / @vue/test-utils	^4.1 / ^2.4	Unit testing
playwright	^1.61	End-to-end testing
oxlint / oxfmt / eslint	—	Linting and formatting

Node requirement: engines field in package.json — ^22.18.0 || >= 24.12.0

Backend

Package	Purpose
Laravel 13 (^13.8) / PHP ^8.3	Application framework
Laravel Sanctum	Token-based API authentication
Eloquent ORM	Database abstraction and relationships
SQLite (development)	Local database (database/database.sqlite)
MySQL (production alternative)	Configured in config/database.php
Gemini API	AI-powered analytics and accounting suggestions (gemini-2.0-flash)
Flysystem S3	Production file storage on Laravel Cloud
Scheduler	Artisan scheduled tasks (daily reports, order cleanup, superadmin password reset, monthly/year-end accounting)

Documentation Tooling

• Python 3 + ReportLab — generates the PDF user manuals and this developer documentation.

• DejaVu Sans fonts — full Swahili character and symbol support in generated PDFs.

• diagrams.net (drawio) — ERD, class, use case, and sequence diagrams; also exported to PDF.

4. Repository Layout

Frontend — erp-electronics/

```
erp-electronics/
├── generate_dev_doc.py # this document generator
├── generate_manual.py # EN + SW user manuals
├── generate_supplier_manual.py # supplier portal manual
├── generate_diagrams.py # drawio diagram generator
├── docs/ # generated PDFs + drawio sources
├── public/ # favicon.svg, index.html, assets
└── src/
├── api/ # axios.js + index.js (API barrel)
├── components/ # shared Vue components
├── composables/ # useTablePagination.js
├── layouts/ # StoreLayout, SuperadminLayout, StoreFooter
├── locales/ # i18n.js, sw.json, en.json
├── pages/ # 53 pages across 13 areas
├── router/ # routes + navigation guard
├── stores/ # auth, business, cart, products, session
├── utils/ # image.js
├── App.vue, main.js
└── __tests__/ # unit tests
```

Backend — erp-electronics-api/

```
erp-electronics-api/
├── app/
│ ├── Console/Commands/ # 5 artisan commands
│ ├── Exceptions/ # AccountingException
│ ├── Http/Controllers/Api/ # 31 controllers
│ ├── Http/Middleware/ # Owner, Superadmin, Supplier
│ ├── Models/ # 38 Eloquent models
│ ├── Providers/ # AppServiceProvider (macros + limiters)
│ ├── Services/ # AccountingEntryService, AccountingReportService, AiSuggestionService
│ └── Support/ # Tenant.php (multi-tenancy resolver)
├── bootstrap/ # app.php (middleware/aliases), providers.php
├── config/ # app, auth, database, filesystems, sanctum, services…
├── database/
│ ├── factories/UserFactory.php
```

```
│ ├── migrations/ # 56 migrations
│ └── seeders/ # DatabaseSeeder, SuperadminSeeder, AccountingSeeder
├── routes/ # api.php (175 routes), web.php, console.php
└── tests/ # Feature + Unit tests
```

5. Frontend Deep Dive — Every File

This chapter documents the purpose and behaviour of every source file in `erp-electronics/`. The stack is Vue 3 (Composition API) with Pinia, Vue Router, Axios, and vue-i18n. Paths are relative to the repository root.

5.1 Application Bootstrap

File	Purpose
`src/main.js`	Application entry point. Creates the Vue app, registers Pinia (with the persistedstate plugin), the router, and i18n. Installs a global helper `$storeLink` that routes links through the business store (slug-aware). Mounts #app.
`src/App.vue`	Root component: renders `<router-view/>` plus the session warning modal. Watches the auth token to start/stop the idle-session timer. Hosts global CSS (Inter font, .btn, .card, .container, form styles, selection color).
`src/api/axios.js`	Single Axios instance. Base URL from `VITE_API_URL` (default localhost:8000/api). Request interceptor injects the Bearer token and the X-Business-Id header. Response interceptor clears the token and redirects to /login on any 401.
`src/api/index.js`	The API barrel — 38 typed modules (authApi, employeeApi, branchApi, ...) that map one-to-one to backend routes. Every page imports its endpoints from here; see §9 for the route mapping.
`src/locales/i18n.js`	Creates the vue-i18n instance (legacy: false). Locale is read from localStorage (key `locale`), defaulting to sw with en as fallback.
`src/locales/sw.json`	Swahili message catalog (hundreds of keys across nav, pages, forms, validation).
`src/locales/en.json`	English message catalog; mirror of sw.json keys.

5.2 Router

File	Purpose
`src/router/index.js`	Client-side router (createWebHistory). Three top-level areas: the storefront at `/` and at `/:businessSlug` (white-label store), the auth pages, and the superadmin panel at `/superadmin`. A global beforeEach guard enforces meta flags: requiresAuth (redirect to /login with redirect query), guest (redirect away from login/register when logged in), and role (fetches the profile if needed and redirects to the user's dashboard when the role does not match).

5.3 Pinia Stores

File	Purpose
src/stores/auth.js	Identity + token. Holds user, token (persisted to localStorage auth_token), and mustChangePassword. Actions: register, login, logout, fetchProfile, updateProfile, changePassword. Role helper computed properties (isCustomer/isEmployee/isOwner/isSuperadmin).
src/stores/business.js	Multi-store context. Holds directory (public business list), mine (owned + co-owned), current business, and slugMode. Persists active_business_id so the Axios interceptor can send X-Business-Id. Provides link() to prefix storefront routes with the active slug.
src/stores/cart.js	Cart state. items, itemCount, subtotal, and a local total that adds a 5000 TSh default shipping constant. Actions fetch/add/update/remove/clear the cart via the API; $reset on logout.
src/stores/products.js	Catalog state: products, featuredProducts, categories, currentProduct, currentCategory, pagination. All fetches append ?business=<slug> when a business is active (businessParams).
src/stores/session.js	Idle-session enforcement: 15-minute inactivity timeout, warning modal for the final 60 seconds, 10-minute grace on tab switch, and a persisted session_last_active timestamp so a reopened tab logs out if the idle window passed. Calls authStore.logout() and routes to /login on terminate.

5.4 Layouts

File	Purpose
src/layouts/StoreLayout.vue	The storefront shell: top bar, search, header actions (cart badge, inbox badge with 15 s unread polling, dashboard link, logout), primary nav with top 5 categories, mobile dropdown, and footer. Applies the business brand colors as CSS variables (--brand, --brand-dark) and the store name in the logo. Mounts the forced-password ChangePasswordModal. Resolves the business context (slug mode vs directory) on mount and when the slug changes.
src/layouts/SuperadminLayout.vue	Shell for the /superadmin panel: sidebar navigation (dashboard, owners, inbox), top bar with the platform name.
src/layouts/StoreFooter.vue	Standalone footer component. Not imported by any current page (the footer is inlined in StoreLayout.vue); retained as a reusable footer for storefront pages.

5.5 Shared Components

File	Purpose
src/components/ChangePasswordModal.vue	Forced password-change dialog shown whenever the API reports must_change_password. Validates strength and calls authApi.changePassword; hides itself on success.

File	Purpose
src/components/ResetOwnerPasswordModal.vue	Superadmin dialog to reset an owner password to the default and optionally unlock the account.
src/components/SessionWarningModal.vue	Countdown overlay for the final minute of the idle session; any activity dismisses it.
src/components/SkeletonLoader.vue	Loading placeholder block used by pages while fetching data.
src/components/TablePagination.vue	Shared pagination bar (per-page count, page links, show-all toggle) bound to the useTablePagination composable.
src/components/product/ProductCard.vue	Storefront product card: image (via imageUrl), name, price, add-to-cart button; used by the product list, home, and search results.

5.6 Composables & Utilities

File	Purpose
src/composables/useTablePagination.js	Client-side pagination + search for tables: filteredItems (dot-path search fields), paginatedItems, pageInfo, goToPage, toggleShowAll. Default 15 per page.
src/utils/image.js	imageUrl(path) helper. Turns relative paths into absolute API URLs: /products/… paths resolve against the API origin derived from VITE_API_URL; already-absolute URLs pass through.

5.7 Pages — Storefront & Customer

Page	Purpose
pages/home/DirectoryPage.vue	Landing page of the platform (route /): lists all active businesses from the public /businesses API as store cards and lets the visitor open a white-label store.
pages/home/HomePage.vue	Storefront homepage of a business (route /:businessSlug): hero, featured products, categories, brand-colored UI.
pages/products/ProductListPage.vue	Searchable, paginated product grid (public /products with ?search, ?category_id, ?business).
pages/products/ProductDetailPage.vue	Product detail: images, variant picker (color/storage), price, stock, add-to-cart.
pages/products/CategoryPage.vue	Category listing with its products and children.
pages/cart/CartPage.vue	Cart review: line items, quantities, subtotal, shipping estimate, proceed to checkout.
pages/checkout/CheckoutPage.vue	Checkout: delivery-required toggle, address selection, shipping-cost calculation (/shipping/calculate), payment provider choice (M-Pesa/Airtel/Mixx/Halopesa/ClickPesa/cash), optional winga reference; places the order and shows the payment outcome.

Page	Purpose
pages/account/OrdersPage.vue	Order history for the logged-in user with status, items, and payments.
pages/account/AccountPage.vue	Profile + address book (CRUD via addressApi) and account settings.
pages/account/SupportPage.vue	Customer support tickets: create a ticket against an order, view replies and status.
pages/customer/CustomerInboxPage.vue	Customer chat with the store owner (conversationApi); live unread badges; WhatsApp-style read ticks (grey = delivered, blue = read) on sent messages.
pages/dashboards/CustomerDashboard.vue	Customer landing after login: quick stats, recent orders, quick links.

5.8 Pages — Authentication

Page	Purpose
pages/auth/LoginPage.vue	Login form with live password-strength rules, remaining-attempts feedback, account-lockout notice, and language switch.
pages/auth/RegisterPage.vue	Customer registration form; enforces the same password rules; creates the account and logs in.

5.9 Pages — Employee & Owner Operations

Page	Purpose
pages/dashboards/EmployeeDashboard.vue	Staff home: stats (orders, revenue, alerts), quick actions, accounting issues scoped to the branch.
pages/dashboards/OwnerDashboard.vue	Owner home: sales KPIs, low-stock alerts, recent orders, quick links, and the AI suggestions panel.
pages/dashboards/analytics/SalesCharts.vue	Chart.js line/bar visuals for monthly sales, items, profit, and cancellations from /analytics/sales.
pages/dashboards/analytics/AiSuggestions.vue	AI insights panel: posts the analytics payload to /analytics/ai-suggestions and renders bilingual suggestions with priority and category.
pages/employee/OrderManagementPage.vue	Order desk (shared owner/employee): filter by status/branch/search; confirm payment, advance status, add tracking, process returns.
pages/employee/CustomerManagementPage.vue	Customer list for staff: search, toggle active, delete.
pages/employee/EmployeeEarningsPage.vue	Employee commissions: pending/paid totals and recent commission records (my-earnings).
pages/employee/EmployeeInboxPage.vue	Employee chat with the owner (owner_employee conversations); read ticks (grey = delivered, blue = read).

Page	Purpose
pages/employee/SupportInboxPage.vue	Staff support-ticket desk: reply, change status, unread/open counts.
pages/owner/ProductManagementPage.vue	Owner product list with search, stock value, active toggle, edit/delete links.
pages/owner/ProductFormPage.vue	Product create/edit form: SKU, prices, category, image upload or URL, variants with per-variant cost/price/quantity.
pages/owner/EmployeeManagementPage.vue	Employee CRUD: NIDA/voting IDs, branch assignment, commission rate, guarantors (Wadhamini), document uploads, reset password.
pages/owner/BranchManagementPage.vue	Branches CRUD with default branch, order/employee counts.
pages/owner/PaymentSettingsPage.vue	Payment providers management and the ClickPesa enable flag.
pages/owner/ShippingSettingsPage.vue	Shipping rules CRUD (from/to city, base cost, value-based tiers).
pages/owner/ReportsPage.vue	Daily and summary sales reports.
pages/owner/InventoryManagementPage.vue	Inventory dashboard: stock levels, adjust (opening/adjustment/damage), transactions log, low-stock list.
pages/owner/PurchaseOrderPage.vue	Purchase orders CRUD and receiving (stock-in + journal).
pages/owner/SupplierManagementPage.vue	Supplier CRUD with legal fields (TIN/VAT/registration), documents, and purchase-order counts.
pages/owner/StockAlertsPage.vue	Low/out-of-stock alerts: acknowledge, resolve.
pages/owner/CommissionManagementPage.vue	Employee commissions: summary per employee, pay individually or pay-all.
pages/owner/OwnerInboxPage.vue	Owner chat: with customers and with the platform superadmin, plus unread counts; read ticks on sent messages.

5.10 Pages — Accounting

Page	Purpose
pages/owner/AccountingDashboardPage.vue	Accounting overview: trial-balance match, recent entries, generated reports, quick links.
pages/owner/ChartOfAccountsPage.vue	Chart of accounts tree with balances; create/edit/delete accounts (system accounts read-only).
pages/owner/JournalEntryListPage.vue	Journal entries list with status filters and search.
pages/owner/JournalEntryCreatePage.vue	Manual journal entry creator: balanced debit/credit line editor, draft or post.

Page	Purpose
pages/owner/JournalEntryDetailPage.vue	Entry detail: lines, totals, poster/void info; post, void, delete actions where allowed.
pages/owner/TrialBalancePage.vue	Trial balance as of a date.
pages/owner/ProfitLossPage.vue	Profit & loss for a date range.
pages/owner/BalanceSheetPage.vue	Balance sheet as of a date.
pages/owner/GeneralLedgerPage.vue	General ledger per account with running balances.

5.11 Pages — Winga Street Promoters

Page	Purpose
pages/winga/WingaManagementPage.vue	Winga CRUD (shared owner/employee): name, phone, TIN/NIDA, commission rate, branch, status.
pages/winga/WingaCommissionPage.vue	Winga commissions list + summary (gross, TRA TDS withholding, net) and payout actions (owner).

5.12 Pages — Superadmin

Page	Purpose
pages/superadmin/SuperadminDashboard.vue	Platform stats: owners, employees, customers, orders, revenue, subscription distribution.
pages/superadmin/OwnerManagementPage.vue	Owners table: create owner, toggle active, delete, navigate to detail.
pages/superadmin/OwnerDetailPage.vue	Owner detail: subscription, limits, password status; reset/set password, force change, unlock account.
pages/superadmin/BrandingPage.vue	White-label branding editor: store name, tagline, logo upload, brand colors.
pages/superadmin/SuperadminInboxPage.vue	Superadmin inbox with owners (superadmin_owner conversations); read ticks on sent messages.

5.13 Pages — Supplier Portal

Page	Purpose
pages/supplier/SupplierPortalPage.vue	Supplier-facing portal: profile, purchase orders assigned to the supplier's email, update PO status (ordered → received).

5.14 Tests

File	Purpose
`src/__tests__/App.spec.js`	Vitest unit test for the root App component (token watch / session wiring).

6. Backend Deep Dive — Every File

This chapter documents the Laravel backend (`erp-electronics-api/`): bootstrap, middleware, providers, models, controllers, services, tenant support, console commands, and configuration. Paths are relative to the backend repository root.

6.1 Bootstrap & Kernel

File	Purpose
`bootstrap/app.php`	Application configuration. withRouting mounts the /api prefix, /up health endpoint, web + console routes. withMiddleware: throttleApi() applies the api rate limiter to every /api route; redirectGuestsTo(null) makes unauthenticated requests return a 401 JSON instead of a redirect; alias() registers the three role middlewares (owner, superadmin, supplier). withExceptions: shouldRenderJsonWhen api/* so every error on API routes is JSON.
`bootstrap/providers.php`	Registers AppServiceProvider as the application provider.
`app/Providers/AppServiceProvider.php`	boot() registers the request macros business() and ownerId() (delegating to Tenant) and the named rate limiters: api (120/min per user id or IP), login (5/min per IP), register (3/min + 10/day per IP).
`app/Exceptions/AccountingException.php`	Domain exception for accounting failures (missing system accounts, unbalanced entries, missing cost prices). Caught by controllers and rendered as HTTP 422.

6.2 Middleware

File	Purpose
`app/Http/Middleware/OwnerMiddleware.php`	Alias owner. Returns 403 "Unauthorized. Owner access required." unless the authenticated user isOwner().
`app/Http/Middleware/SuperadminMiddleware.php`	Alias superadmin. Returns 403 "Unauthorized" unless isSuperadmin().
`app/Http/Middleware/SupplierMiddleware.php`	Alias supplier. Requires an authenticated, is_active user (else 401); then requires role supplier or superadmin (else 403 "Forbidden"). Wraps the supplier-portal routes.

6.3 Models (app/Models/) — 38

Model	Table	Purpose & key behaviour
User	users	Identity for all roles (customer/employee/owner/supplier/superadmin). Attributes: role enum, is_active, is_superadmin, phone, password_changed_at, failed_login_attempts, locked_until. Constants MAX_LOGIN_ATTEMPTS=5, LOCKOUT_MINUTES=30. Logic: mustChangePassword() (≥3 days), superadminPasswordExpired() (≥6 months), getPasswordStatus(), recordFailedLogin/resetFailedLoginAttempts/unlockAccount/isLocked. Relations: profiles, orders, addresses, documents, guarantors, branches, businesses.
OwnerProfile	owner_profiles	Subscription + white-label branding per owner: plan, expiry, max_products/max_employees limits, brand_store_name/tagline/logo/colors. Helpers isSubscriptionActive(), isTrialActive().
EmployeeProfile	employee_profiles	Employment data: employee_code (EMP-xxxxxx), position, department, hire_date, branch_id, commission_rate, base_salary, NIDA/voting ID numbers.
CustomerProfile	customer_profiles	Customer extras: date_of_birth, loyalty_points.
Category	categories	Self-referential parent/children tree; name_sw for Swahili; getTranslatedNameAttribute() respects Accept-Language: sw.
Product	products	SKU, name, slug (auto from name on creating), price, cost_price, images JSON, category_id, owner_id. Cost price feeds COGS.
ProductVariant	product_variants	Variant per product: color/storage/SKU, own price + cost_price; hasOne inventory.
Inventory	inventory	quantity_on_hand + reorder_level per variant; isInStock()/needsReorder().
Address	addresses	Customer/owner address book rows (label, street, city, is_default).
Order	orders	status machine (pending_payment/pending/inactive/paid/processing/shipped/delivered/cancelled); order_number auto ORD-xxxx; subtotal/shipping/total/winga_fee; branch_id, handled_by, delivery fields. markAsPaid() sets paid and decrements inventory.
OrderItem	order_items	Line item: variant, quantity, unit_price, total, returned_quantity.
Payment	payments	Provider (mpesa/airtel/mixx_by_yas/halopesa/clickpesa/cash), amount, status (pending/completed/failed), metadata, provider_reference.
PaymentProvider	payment_providers	Enabled payment method configuration (name, slug, number, icon, sort).

Model	Table	Purpose & key behaviour
Branch	branches	Owner branch; tenant boundary for orders/employees; is_default.
Conversation	conversations	Message threads: type (superadmin_owner/customer_owner/owner_employee), participants, status; otherParty() resolves the counterpart.
ConversationMessage	conversation_messages	Message rows with is_read (read receipts); conversation + sender relations. ConversationController::show() flips is_read when the recipient opens the thread.
SupportMessage	support_messages	Customer support tickets (category, status, admin_reply).
ShippingRule	shipping_rules	City-pair shipping cost with value-based tiers; calculateCost() picks the first matching tier.
Setting	settings	Key/value store (type-tagged) for vat_rate, income_tax_rate, winga_wht_rate, clickpesa_enabled, prices_include_vat. getTypedValue() decodes by type.
DailyReport	daily_reports	Generated daily sales snapshot (revenue, items, paid/pending/cancelled, employee_stats, top_products).
Account	accounts	Chart of accounts: code (unique per owner), type, normal_balance, parent tree, is_system. getBalanceAttribute() sums posted lines by normal-balance direction.
JournalEntry	journal_entries	Double-entry header: reference JE-YYYY-seq, date, status (draft/posted/voided), source_type/source_id, prepared_by/posted_by/voided_by. isBalanced() checks |debit-credit| < 0.01.
JournalLine	journal_lines	Entry lines: account_id, debit, credit, description.
AccountingReport	accounting_reports	Generated monthly/yearly statements with data + summary JSON, suggestions JSON, is_finalized.
Commission	commissions	Employee commission per paid order (order_amount, cost_amount, profit_amount, rate, amount; status pending/paid/reversed).
InventoryTransaction	inventory_transactions	Stock movement audit (sale/return/adjustment/purchase/damage/opening) with quantity_after and reference.
Supplier	suppliers	Vendor master: legal fields (business_type, TIN, VAT, registration), contact, address; documents relation.
PurchaseOrder	purchase_orders	PO header: po_number PO-YYYY-seq, status (draft/ordered/received), total_cost, dates, journal_entry_id.
PurchaseOrderItem	purchase_order_items	PO line items with quantity_received.

Model	Table	Purpose & key behaviour
StockAlert	stock_alerts	Low/out-of-stock alerts (active/acknowledged/resolved).
EmployeeDocument	employee_documents	Uploaded employee files (contract/background_check/other).
EmployeeGuarantor	employee_guarantors	Guarantor records (Wadhamini) per employee.
SupplierDocument	supplier_documents	Uploaded supplier legal documents (8 categories).
Notification	notifications	In-app notifications (custom table, not Laravel's): type, title, message, link, read_at.
Business	businesses	White-label store per owner: name, slug, tagline, logo, is_active; relations to members (co-owners) and products.
BusinessUser	business_user	Pivot (composite PK business_id+user_id) with role column for co-ownership.
Winga	wingas	Street promoter: phone, TIN/NIDA, commission_rate (%), branch, status.
WingaCommission	winga_commissions	Per-order winga commission: gross, withholding_tax (TRA TDS), net, status (pending/paid/reversed), journal_entry_id.

6.4 Controllers (app/Http/Controllers/Api/) — 31

Controller	Area	Responsibilities
AuthController	Auth	register (public, creates customer), login (lockout + attempts + flags), logout, profile, updateProfile, changePassword.
ProductController	Catalog	Public index/show/featured (business-scoped) + owner store/update/destroy/manage with variant + inventory creation and image handling.
CategoryController	Catalog	Public category tree + detail with active products; translated names.
BusinessController	Multi-store	Public index/by-slug; owner mine; presentation of store_name/colors.
CartController	Commerce	Cart-as-pending_payment-order: index/add/update/remove/clear with stock checks and recalculation.
OrderController	Commerce	Checkout store, index/show, manage, updateStatus (lifecycle + auto-journaling), updateDelivery, returnItems (partial returns).
PaymentController	Commerce	initiate (auto-confirms ClickPesa/cash), webhook (no auth), status.
PaymentProviderController	Commerce	Public enabled list + owner/staff management CRUD.

Controller	Area	Responsibilities
ShippingController	Commerce	Public calculate (city pairs + wildcards) + owner CRUD of rules.
AddressController	Commerce	Authenticated user address book (apiResource).
BranchController	Tenant	Owner branches CRUD + set-default.
EmployeeController	HR	Owner-only employee CRUD: default password = strtoupper(name), branch assignment, NIDA/voting IDs, guarantors, document upload/download, reset-password, toggle-status.
CustomerController	Customers	Staff customer list, toggle-status, destroy.
ReportController	Reports	daily + summary reports; generateForDate builds the snapshot.
AnalyticsController	Analytics	sales (SQLite strftime grouping, zero-filled months, summary metrics) + ai-suggestions (Gemini with fallback).
ConversationController	Messaging	Role-based conversations: index/store/show/sendMessage/updateStatus/unreadCount/contacts/ownerDetails/customerDetails. show() marks incoming messages read (read receipts), powering the frontend WhatsApp-style ticks.
SupportMessageController	Support	Tickets: index/store/show/reply/updateStatus/unreadCount.
NotificationController	Notifications	index/count/markRead/markAllRead + static create factory.
AccountController	Accounting	Chart of accounts index/tree/store/update/destroy with system-account protections.
JournalEntryController	Accounting	Manual entries: index/store (balanced draft)/show/update/post/void/destroy.
AccountingReportController	Accounting	Live statements (trial-balance/profit-loss/balance-sheet/general-ledger) + generate monthly/yearly + list/show + ai-suggestions.
AccountingIssuesController	Accounting	Actionable issue buckets (unconfirmed payments, pending deliveries, missing cost prices, drafts, voids, pending commissions, unbalanced trial balance, low stock) with owner vs branch scoping.
CommissionController	Payroll	Commissions index/summary/pay/pay-all/employee-earnings with payout journals.
WingaController	Winga	Winga CRUD + toggle-status (owner/employee, tenant-scoped).
WingaCommissionController	Winga	Commissions index/summary/pay/pay-all with TRA TDS payout journals.

Controller	Area	Responsibilities
InventoryController	Inventory	index/adjust/transactions/low-stock/dashboard; adjustment posts inventory journals + triggers low-stock check.
PurchaseOrderController	Procurement	PO index/store/show/receive/destroy + supplier-portal supplierOrders/supplierShow/supplierUpdateStatus.
SupplierController	Procurement	Supplier CRUD + documents + portal profile.
StockAlertController	Inventory	index/count/acknowledge/resolve + static checkLowStock used by other flows.
SettingsController	Platform	Public payment flag + branding; owner updatePayment.
SuperadminController	Platform	Stats, owner CRUD, subscription/limits/branding/logo, password status/reset/set/force/unlock, all-password status.

6.5 Services

File	Purpose
`app/Services/AccountingEntryService.php`	The accounting engine. VAT/WHT helpers (getVatRate default 18, pricesIncludeVat default true, getWingaWhtRate default 5, splitVat). postSale() posts cash/revenue/VAT/COGS/winga accrual; reverseSale() reverses it; postReturn() books partial returns; postInventoryAdjustment() books stock adjustments; createCommission()/createWingaCommission() accrue payables; reverseCommissions()/reverseWingaCommissions() handle cancellation clawbacks; adjustCommissionForReturn()/adjustWingaCommissionForReturn() prorate on partial returns; closeYear() posts the year-end entry to retained earnings.
`app/Services/AccountingReportService.php`	computeTrialBalance/computeProfitLoss/computeBalanceSheet (posted entries only) and generateMonthlyReport/generateYearlyReport with data+summary JSON persistence.
`app/Services/AiSuggestionService.php`	Bilingual (EN/SW) accounting suggestions from Gemini (gemini-2.0-flash, temperature 0.7, 4096 tokens, 60 s timeout) with deterministic rule-based fallback (M-Pesa reconciliation, VAT/TRA reminders, inventory counts).

6.6 Multi-Tenancy Support

File	Purpose
app/Support/Tenant.php	activeBusiness(request) resolves the current business for an owner (owned + co-owned, X-Business-Id first). ownerId(request) returns the active business owner_id or the user's own id. forUser() lists manageable businesses; bySlug() resolves a storefront business.

6.7 Console Commands (app/Console/Commands/) — 5

Command	Purpose
GenerateDailyReport (report:daily)	Generates the daily sales report for --date (default yesterday).
CleanupUnpaidOrders (orders:cleanup-unpaid)	Marks pending/pending_payment orders inactive after 3 h and deletes inactive orders after 6 h.
ResetSuperadminPassword (superadmin:reset-password)	Resets the superadmin password to SuperAdmin@2026 when older than 6 months.
GenerateAccountingReports (accounting:generate-reports)	Generates monthly reports for active owners; --with-suggestions also runs the AI suggestions.
CloseYearAccounting (accounting:close-year)	Posts year-end closing entries into retained earnings for all active owners.

6.8 Scheduler (routes/console.php)

Schedule	Task
Daily 00:10	report:daily
Every 5 minutes	orders:cleanup-unpaid
Monthly	superadmin:reset-password
1st of month 00:30	accounting:generate-reports --with-suggestions
Jan 1 01:00	accounting:close-year

6.9 Configuration Highlights (config/)

File	Notable values
app.php	timezone UTC, locale en (env-overridable), cipher AES-256-CBC.
auth.php	web guard with users provider; password broker on password_reset_tokens (60 min).
database.php	Default connection sqlite; full mysql/mariadb/pgsql/sqlsrv blocks; MySQL utf8mb4 strict.
filesystems.php	Default disk local; public disk; s3 disk for Laravel Cloud (FILESYSTEM_DISK=s3).

File	Notable values
sanctum.php	Stateful domains include localhost:5173/5174 and the app URL; tokens never expire by default.
services.php	gemini block reading GEMINI_API_KEY.
session.php	database driver, 120 min lifetime, http_only, same_site lax.
cache.php	database store by default.
queue.php	database connection, retry_after 90 s.
mail.php	log mailer by default.
logging.php	stack → single channel at debug level.

7. Database Schema & Models

The database contains 46 business tables plus Laravel framework tables, built by 56 migrations. A full entity-relationship diagram is available as ERD.drawio (crow's foot notation). This section lists the tables by domain and the notable columns added by later migrations.

Tables by Domain

Domain	Tables
Identity	users, customer_profiles, employee_profiles, owner_profiles, personal_access_tokens
Multi-store	businesses, business_user
Catalog	categories, products, product_variants, inventory
Commerce	addresses, branches, orders, order_items, payments, payment_providers, shipping_rules
Support	support_messages, conversations, conversation_messages, notifications
Accounting	accounts, journal_entries, journal_lines, accounting_reports
Payroll	commissions, wingas, winga_commissions
Operations	inventory_transactions, purchase_orders, purchase_order_items, suppliers, stock_alerts
Documents	employee_documents, employee_guarantors, supplier_documents
Settings	settings, daily_reports
Framework	password_reset_tokens, sessions, cache, cache_locks, jobs, job_batches, failed_jobs

Order Status Enum

The `orders.status` enum evolved across migrations to its current value set:

Value set	pending, pending_payment, inactive, paid, processing, shipped, delivered, cancelled
Default	pending_payment (a cart)
Meaning	pending_payment = active cart; pending = checked out awaiting payment confirmation; inactive = abandoned (cleanup command); paid = confirmed (inventory + journals); then processing → shipped → delivered; cancelled is terminal; returns may auto-cancel fully-returned orders

Key Relationships

• User has one profile row depending on role (customer_profile / employee_profile / owner_profile) and belongs to businesses via the business_user pivot (co-owners).

• Product → category, has many variants → one inventory row each. Products/categories/shipping_rules carry owner_id.

• Order → user, optional branch, winga (plain column), items, payments, shipping address; handled_by references the staff user.

• Conversation → owner, optional customer/employee/superadmin; has many messages.

• JournalEntry → owner, prepared/posted/voided by users; has many journal_lines → accounts.

• Commission / WingaCommission → owner, order, journal entry; commissions reference the employee.

• PurchaseOrder → owner, supplier (nullable), items → variants; received POs post inventory journals.

• Business → owner; products and members relate back to the business (products by owner_id; members via pivot).

System Account Codes

Code	Name	Type
1020	Cash (M-Pesa/Bank) — cash base used by payouts	Asset
1200	Inventory	Asset
2100	Winga Commission Payable	Liability
2120	Withholding Tax Payable (TDS)	Liability
2500	VAT Output	Liability
3010	Owner's Capital	Equity
3020	Retained Earnings	Equity
4010	Sales Revenue	Revenue
4020	Shipping Revenue	Revenue
5010	Cost of Goods Sold	Expense
5100	Inventory Adjustments	Expense
5110	Commission Expense	Expense

8. Authentication & Authorization

Token Flow

1. `POST /api/auth/login → { email, password }`
2. `Backend validates credentials and returns:`

`{ token, user, must_change_password, superadmin_password_expired }`

3. `Frontend stores the token in localStorage (key: auth_token)`
4. `Every request sends: Authorization: Bearer <token>`
5. `A 401 response clears the token and redirects to /login`

Password Policy

- Minimum 8 characters with at least one uppercase, one lowercase, one number, and one special character.
- Enforced on both the frontend (real-time rules) and the backend (custom validation messages).
- Password expiry: `password_changed_at` must be newer than 3 days, otherwise a forced change modal appears on login (owners).

Default Passwords

Employee (owner-created)	Full name in capitals — e.g. "MATHEW ZACHARIA"; password_changed_at null forces a change
Owner (superadmin-created)	Full name in capitals + random 3-digit suffix — e.g. "JOHN DOE123"; forced change on first login
Superadmin	SuperAdmin@2026 — auto-reset every 6 months via scheduled artisan command

Account Lockout

After 5 failed login attempts the account locks for 30 minutes (`users.locked_until`). The API returns HTTP 423 with a remaining-minutes message; the login form also reports remaining attempts.

Role Enforcement

Roles gate both routes (backend middleware) and navigation (frontend router meta). The middleware chain per request is: api group (throttle:api) → auth:sanctum → optional owner / superadmin / supplier role middleware. Missing tokens return 401 JSON; role violations return 403 JSON with a distinct message per middleware.

Session Idle Timeout (frontend)

• 15 minutes of inactivity → session ends (mouse, keyboard, touch, scroll, click, wheel reset the timer).

• At 14 minutes a warning modal shows a 60-second countdown; any activity dismisses it.

• Switching tabs/apps pauses the timer; returning within 10 minutes continues the session, otherwise the user is signed out.

• A `session_last_active` timestamp is persisted, so returning to a closed tab after the idle window forces logout on load.

• Logout clears `auth_token` and revokes the Sanctum token on the server.

9. API Reference — All 175 Routes

All endpoints are JSON and mounted under the `/api` prefix. Development base: `http://localhost:8000/api`. Production base: `https://api.electroshophub.online/api`. Of the 175 routes, 14 are public and 161 require a Bearer token; 35 are gated by the owner middleware, 16 by superadmin, and 4 by supplier. The only named routes are the 5 auto-generated addresses.* routes.

9.1 Public Endpoints (no auth)

Method	URI	Controller@method	Ext. middleware
POST	/auth/register	AuthController@register	throttle:register
POST	/auth/login	AuthController@login	throttle:login
GET	/products	ProductController@index	—
GET	/products/featured	ProductController@featured	—
GET	/products/{slug}	ProductController@show	—
GET	/businesses	BusinessController@index	—
GET	/businesses/by-slug/{slug}	BusinessController@show	—
GET	/categories	CategoryController@index	—
GET	/categories/{slug}	CategoryController@show	—
GET	/payment-providers	PaymentProviderController@publicIndex	—
POST	/payments/webhook	PaymentController@webhook	—
POST	/shipping/calculate	ShippingController@calculate	—
GET	/settings/payment	SettingsController@payment	—
GET	/settings/branding	SettingsController@branding	—

9.2 Authenticated — Auth, Businesses, Branches, Employees

Method	URI	Controller@method	Role
POST	/auth/logout	AuthController@logout	—
GET	/auth/profile	AuthController@profile	—
PUT	/auth/profile	AuthController@updateProfile	—
POST	/auth/change-password	AuthController@changePassword	—
GET	/businesses/mine	BusinessController@mine	—

Method	URI	Controller@method	Role
GET	/branches	BranchController@index	—
POST	/branches	BranchController@store	—
GET	/branches/{branch}	BranchController@show	—
PUT	/branches/{branch}	BranchController@update	—
PATCH	/branches/{branch}/set-default	BranchController@setDefault	—
DELETE	/branches/{branch}	BranchController@destroy	—
GET	/employees	EmployeeController@index	owner
POST	/employees	EmployeeController@store	owner
PUT	/employees/{user}	EmployeeController@update	owner
PATCH	/employees/{user}/toggle-status	EmployeeController@toggleStatus	owner
PATCH	/employees/{user}/assign-branch	EmployeeController@assignBranch	owner
DELETE	/employees/{user}	EmployeeController@destroy	owner
PUT	/employees/{user}/profile	EmployeeController@updateProfile	owner
POST	/employees/{user}/reset-password	EmployeeController@resetPassword	owner
GET	/employees/{user}/documents	EmployeeController@indexDocuments	owner
POST	/employees/{user}/documents	EmployeeController@storeDocuments	owner
DELETE	/employees/{user}/documents/{document}	EmployeeController@destroyDocument	owner
GET	/employees/{user}/documents/{document}/download	EmployeeController@downloadDocument	owner

9.3 Authenticated — Customers, Products, Settings, Payments

Method	URI	Controller@method	Role
GET	/customers	CustomerController@index	staff
PATCH	/customers/{user}/toggle-status	CustomerController@toggleStatus	staff
DELETE	/customers/{user}	CustomerController@destroy	staff
GET	/products-manage	ProductController@manage	owner
POST	/products	ProductController@store	owner
PUT	/products/{id}	ProductController@update	owner

Method	URI	Controller@method	Role
DELETE	/products/{id}	ProductController@destroy	owner
PUT	/settings/payment	SettingsController@updatePayment	owner
GET	/payment-providers-manage	PaymentProviderController@index	—
POST	/payment-providers	PaymentProviderController@store	—
PUT	/payment-providers/{id}	PaymentProviderController@update	—
DELETE	/payment-providers/{id}	PaymentProviderController@destroy	—
POST	/payments/initiate	PaymentController@initiate	—
GET	/orders/{orderId}/payment-status	PaymentController@status	—

9.4 Authenticated — Cart, Orders, Addresses, Support

Method	URI	Controller@method	Role
GET	/cart	CartController@index	—
POST	/cart	CartController@add	—
PUT	/cart/{itemId}	CartController@update	—
DELETE	/cart/{itemId}	CartController@remove	—
DELETE	/cart	CartController@clear	—
GET	/orders	OrderController@index	—
POST	/orders	OrderController@store	—
GET	/orders/{orderId}	OrderController@show	—
GET	/orders-manage	OrderController@manage	staff
PATCH	/orders/{orderId}/status	OrderController@updateStatus	staff
POST	/orders/{orderId}/return	OrderController@returnItems	staff
PATCH	/orders/{orderId}/delivery	OrderController@updateDelivery	staff
GET	/addresses	AddressController@index	—
POST	/addresses	AddressController@store	—
GET	/addresses/{address}	AddressController@show	—
PUT/PATCH	/addresses/{address}	AddressController@update	—
DELETE	/addresses/{address}	AddressController@destroy	—
GET	/support-messages	SupportMessageController@index	—
POST	/support-messages	SupportMessageController@store	—

Method	URI	Controller@method	Role
GET	/support-messages/{id}	SupportMessageController@show	—
PATCH	/support-messages/{id}/reply	SupportMessageController@reply	staff
PATCH	/support-messages/{id}/status	SupportMessageController@updateStatus	—
GET	/support/unread-count	SupportMessageController@unreadCount	—

9.5 Authenticated — Shipping, Analytics, Conversations

Method	URI	Controller@method	Role
GET	/shipping-rules	ShippingController@index	owner
POST	/shipping-rules	ShippingController@store	owner
PUT	/shipping-rules/{id}	ShippingController@update	owner
DELETE	/shipping-rules/{id}	ShippingController@destroy	owner
GET	/analytics/sales	AnalyticsController@sales	owner/staff
POST	/analytics/ai-suggestions	AnalyticsController@aiSuggestions	owner
GET	/conversations	ConversationController@index	—
POST	/conversations	ConversationController@store	—
GET	/conversations/unread-count	ConversationController@unreadCount	—
GET	/conversations/contacts	ConversationController@contacts	—
GET	/conversations/{conversation}	ConversationController@show	—
POST	/conversations/{conversation}/messages	ConversationController@sendMessage	—
PATCH	/conversations/{conversation}/status	ConversationController@updateStatus	—
GET	/conversations/{conversation}/owner-details	ConversationController@ownerDetails	—
GET	/conversations/{conversation}/customer-details	ConversationController@customerDetails	—

Read receipts: every message carries an is_read flag. GET /conversations/{conversation} marks all incoming messages from the requesting user's counterpart as read; the sender picks this up via the 15 s polling in the inbox pages and renders grey (delivered) vs blue (read) ticks. There is no separate delivery flag — messages are server-stored, so unread is the only distinguishable state.

9.6 Authenticated — Accounting (owner)

Method	URI	Controller@method
GET	/accounts	AccountController@index
GET	/accounts/tree	AccountController@tree
POST	/accounts	AccountController@store
PUT	/accounts/{id}	AccountController@update
DELETE	/accounts/{id}	AccountController@destroy
GET	/journal-entries	JournalEntryController@index
POST	/journal-entries	JournalEntryController@store
GET	/journal-entries/{id}	JournalEntryController@show
PUT	/journal-entries/{id}	JournalEntryController@update
DELETE	/journal-entries/{id}	JournalEntryController@destroy
POST	/journal-entries/{id}/post	JournalEntryController@post
POST	/journal-entries/{id}/void	JournalEntryController@void
GET	/reports/trial-balance	AccountingReportController@trialBalance
GET	/reports/profit-loss	AccountingReportController@profitLoss
GET	/reports/balance-sheet	AccountingReportController@balanceSheet
GET	/reports/general-ledger	AccountingReportController@generalLedger
POST	/reports/generate-monthly	AccountingReportController@generateMonthly
POST	/reports/generate-yearly	AccountingReportController@generateYearly
GET	/reports/list	AccountingReportController@listReports
GET	/reports/{id}	AccountingReportController@showReport
POST	/reports/ai-suggestions	AccountingReportController@aiSuggestions
GET	/accounting-issues	AccountingIssuesController@index

9.7 Authenticated — Reports, Commissions, Wingas, Inventory, Procurement

Method	URI	Controller@method	Role
GET	/reports/daily	ReportController@daily	staff
GET	/reports/summary	ReportController@summary	staff
GET	/commissions	CommissionController@index	owner/staff

Method	URI	Controller@method	Role
GET	/commissions/summary	CommissionController@summary	owner/staff
POST	/commissions/{id}/pay	CommissionController@pay	owner
POST	/commissions/pay-all	CommissionController@payAll	owner
GET	/commissions/my-earnings	CommissionController@employeeEarnings	employee
GET	/wingas	WingaController@index	owner/staff
POST	/wingas	WingaController@store	owner/staff
PUT	/wingas/{winga}	WingaController@update	owner/staff
PATCH	/wingas/{winga}/toggle-status	WingaController@toggleStatus	owner/staff
DELETE	/wingas/{winga}	WingaController@destroy	owner/staff
GET	/winga-commissions	WingaCommissionController@index	owner/staff
GET	/winga-commissions/summary	WingaCommissionController@summary	owner/staff
POST	/winga-commissions/{id}/pay	WingaCommissionController@pay	owner
POST	/winga-commissions/pay-all	WingaCommissionController@payAll	owner
GET	/inventory	InventoryController@index	owner
POST	/inventory/adjust	InventoryController@adjust	owner/staff
GET	/inventory/transactions	InventoryController@transactions	owner
GET	/inventory/low-stock	InventoryController@lowStock	owner
GET	/inventory/dashboard	InventoryController@dashboard	owner
GET	/purchase-orders	PurchaseOrderController@index	owner
POST	/purchase-orders	PurchaseOrderController@store	owner
GET	/purchase-orders/{id}	PurchaseOrderController@show	owner
POST	/purchase-orders/{id}/receive	PurchaseOrderController@receive	owner
DELETE	/purchase-orders/{id}	PurchaseOrderController@destroy	owner
GET	/suppliers	SupplierController@index	owner
GET	/suppliers/all	SupplierController@all	owner
POST	/suppliers	SupplierController@store	owner

Method	URI	Controller@method	Role
GET	/suppliers/{id}	SupplierController@show	owner
PUT	/suppliers/{id}	SupplierController@update	owner
DELETE	/suppliers/{id}	SupplierController@destroy	owner
GET	/suppliers/{id}/documents	SupplierController@indexDocuments	owner
POST	/suppliers/{id}/documents	SupplierController@storeDocumentsForSupplier	owner
DELETE	/suppliers/{id}/documents/{document}	SupplierController@destroyDocument	owner
GET	/suppliers/{id}/documents/{document}/download	SupplierController@downloadDocument	owner
GET	/stock-alerts	StockAlertController@index	owner
GET	/stock-alerts/count	StockAlertController@count	owner
POST	/stock-alerts/{id}/acknowledge	StockAlertController@acknowledge	owner
POST	/stock-alerts/{id}/resolve	StockAlertController@resolve	owner
GET	/notifications	NotificationController@index	—
GET	/notifications/count	NotificationController@count	—
POST	/notifications/{id}/read	NotificationController@markRead	—
POST	/notifications/read-all	NotificationController@markAllRead	—

9.8 Supplier Portal (supplier middleware)

Method	URI	Controller@method
GET	/supplier-portal/profile	SupplierController@supplierProfile
GET	/supplier-portal/purchase-orders	PurchaseOrderController@supplierOrders
GET	/supplier-portal/purchase-orders/{id}	PurchaseOrderController@supplierShow
POST	/supplier-portal/purchase-orders/{id}/update-status	PurchaseOrderController@supplierUpdateStatus

9.9 Superadmin (superadmin middleware)

Method	URI	Controller@method
GET	/superadmin/stats	SuperadminController@stats

Method	URI	Controller@method
GET	/superadmin/owners	SuperadminController@index
POST	/superadmin/owners	SuperadminController@store
GET	/superadmin/owners/{id}	SuperadminController@show
PATCH	/superadmin/owners/{id}/toggle-active	SuperadminController@toggleActive
PUT	/superadmin/owners/{id}/subscription	SuperadminController@updateSubscription
PUT	/superadmin/owners/{id}/limits	SuperadminController@updateLimits
PUT	/superadmin/owners/{id}/branding	SuperadminController@updateBranding
POST	/superadmin/owners/{id}/branding-logo	SuperadminController@updateBrandingLogo
DELETE	/superadmin/owners/{id}	SuperadminController@destroy
GET	/superadmin/passwords/status	SuperadminController@allPasswordsStatus
GET	/superadmin/owners/{id}/password-status	SuperadminController@getPasswordStatus
POST	/superadmin/owners/{id}/reset-password	SuperadminController@resetPassword
POST	/superadmin/owners/{id}/set-password	SuperadminController@setPassword
POST	/superadmin/owners/{id}/force-password-change	SuperadminController@forcePasswordChange
POST	/superadmin/owners/{id}/unlock-account	SuperadminController@unlockAccount

Rate Limits

Limiter	Limit	Keyed by
api (global)	120 requests per minute	Authenticated user ID, else IP
login	5 requests per minute	IP address
register	3 per minute and 10 per day	IP address

Exceeded limits return HTTP 429 with X-RateLimit-Remaining / X-RateLimit-Reset headers.

10. Superadmin Module

The superadmin is the platform administrator responsible for onboarding and managing business owners. Access the panel at `/superadmin` (role-guarded). This module is intentionally not described in the end-user manuals.

10.1 System Overview

- System Statistics — total customers, total orders, total revenue, and active owners at a glance.
- Owners Table — all registered owners with company name, plan, status, and registration date.

10.2 Managing Owners

- Create Owner — registers an owner with name, email, phone, and company name. Default password is the full name in capitals plus a random 3-digit suffix; the response exposes `default_password` to share securely.
- Toggle Active/Inactive — enable or disable an owner's account instantly.
- Delete Owner — removes the owner and all associated data.

10.3 Owner Details

Opening an owner reveals:

- Subscription — plan (free/starter/pro/enterprise), status, and expiry date.
- Limits — maximum products and employees the owner may register.
- Branding — white-label configuration: store name, tagline, logo upload, and the two brand colors.

10.4 Owner Password Management

- Reset Password — resets to the default and forces a change on next login.
- Set Password — assigns an explicit password.
- Force Password Change — clears `password_changed_at` so the owner must change it.
- Unlock Account — clears `locked_until` after a 30-minute lockout.
- Passwords Status — overview of every owner's password state (changed / needs change / expired).

10.5 Superadmin Inbox

The superadmin communicates with owners through the Inbox (owner ↔ superadmin conversations). Conversations appear in real time with unread badges.

10.6 Superadmin Password Lifecycle

The default superadmin password is SuperAdmin@2026. A scheduled command (`php artisan superadmin:reset-password`) auto-resets it every 6 months; login and profile responses expose `superadmin_password_expired` so the UI can prompt for a change.

11. Security Measures

11.1 API Rate Limiting

Laravel's named rate limiters protect every API route. The api group limiter is attached to the whole `/api/*` group; the login and register limiters sit on the public auth routes.

```
// app/Providers/AppServiceProvider.php
RateLimiter::for('api', fn (Request $r) =>
Limit::perMinute(120)->by($r->user()?->id ?: $r->ip()));
RateLimiter::for('login', fn (Request $r) => Limit::perMinute(5)->by($r->ip()));
RateLimiter::for('register', fn (Request $r) => [
Limit::perMinute(3)->by($r->ip()),
Limit::perDay(10)->by($r->ip()),
]);
```

Registered with $middleware->throttleApi() in bootstrap/app.php and throttle:login / throttle:register on the auth routes.

11.2 Passwords & Lockout

- Complexity enforced at the API level (8+ chars; upper, lower, number, symbol).
- Accounts lock after 5 failed attempts for 30 minutes.
- Passwords older than 3 days trigger a mandatory change; superadmin password resets every 6 months.

11.3 Tenant Isolation

- Every owner-scoped controller resolves the tenant through `request()->ownerId()` and scopes all queries to it.
- Cross-tenant access attempts fail fast: owner-scoped findOrFail returns 404, guard checks return 403.
- Employees are resolved to their branch owner before any owner-scoped data is served.

11.4 File Uploads

- Employee documents accept only PDF, JPG, PNG, DOC, DOCX, up to 20 MB each; stored under `employee-documents/`.
- Supplier documents and product/branding images have per-type size limits and category validation.
- Downloads are served only through authenticated, owner-scoped endpoints.

11.5 JSON Error Rendering

The `shouldRenderJsonWhen api/*` rule guarantees every API error — validation 422s, 401/403, 404s, 429s, and 500s — is returned as JSON, so the SPA always receives parseable responses.

12. Analytics & AI Insights

Sales Analytics

`AnalyticsController::sales()` accepts `?months=12` and computes, in parallel: monthly sales, monthly items sold, monthly profit, and monthly cancellations. It returns a gap-free month list plus category_breakdown, top_products, and a summary (total revenue, profit, orders, items sold, average order value, profit margin, revenue and order growth).

AI Suggestions

The owner dashboard can call `POST /analytics/ai-suggestions` with the sales payload. The backend builds a Tanzania-specific business prompt and calls Gemini 2.0 Flash (temperature 0.7, max 2048 tokens, configured via `GEMINI_API_KEY`). If the AI response is unavailable, rule-based fallback suggestions are returned.

Suggestions are bilingual (`title_sw/title_en`, `description_sw/description_en`) with a priority (high/medium/low), category (inventory/pricing/marketing/growth/operations), expected impact, and a `source` of "ai" or "fallback".

Accounting AI Suggestions

The accounting module has a second Gemini integration (`AiSuggestionService`): after generating a monthly report, owners can request up to 8 bilingual accounting suggestions (revenue/expenses/compliance/cash flow/inventory/tax/growth) with prior-period trend context. Results persist on `accounting_reports.suggestions`.

13. End-to-End Request Cycles

This chapter walks through the complete journeys through both codebases, from a page action in the SPA to the database and back — and the auto-journaling the system performs at each step.

13.1 Generic Request Pipeline

```
Browser (Vue page) → store action → src/api module → axios instance
→ interceptor adds Bearer token + X-Business-Id → HTTP request
→ public/index.php → bootstrap/app.php → router (/api prefix)
→ throttle:api → auth:sanctum (Bearer token) → role middleware
→ Controller → validates → Service (accounting) → Eloquent models
→ SQLite (dev) / MySQL (prod) → JSON response
→ axios response interceptor → page renders; 401 → clear token → /login
```

13.2 Authentication Cycle

- Login: LoginPage → authStore.login() → authApi.login() → POST /auth/login → AuthController@login validates, checks lockout (423) and attempts, verifies the hash, issues a Sanctum token, eager-loads role relations, returns { token, user, must_change_password, superadmin_password_expired }. The store persists only the token.
- Forced change: if must_change_password, StoreLayout mounts ChangePasswordModal → POST /auth/change-password → updates the hash + password_changed_at.
- Restore: on any page load the router guard calls authStore.fetchProfile() → GET /auth/profile when a token exists.

13.3 Storefront Browse Cycle

- Directory: DirectoryPage → businessApi.list() → GET /businesses (public) → BusinessController@index presents each active business with store_name, brand colors, and product counts.
- Open a store: the URL becomes /:businessSlug → StoreLayout resolves the business via businessStore.loadBySlug() (GET /businesses/by-slug/{slug}) and applies branding as CSS variables.
- Catalog: HomePage/ProductListPage → productStore.fetchProducts() → GET /products?business=<slug> → ProductController@index scopes by Tenant::bySlug and returns paginated products with variants + inventory.
- Detail: ProductDetailPage → GET /products/{slug} → variant picker and stock read from the response.

13.4 Cart Cycle

The cart is simply an Order row with status pending_payment. CartController::getOrCreateCart() does firstOrCreate per user. Every cart mutation recalculaates subtotal and total server-side and

returns the fresh items.

- Add: CartPage/ProductCard → cartStore.addItem() → POST /cart { product_variant_id, quantity } → stock check (422 "Insufficient stock") → line added or quantity increased.
- Update/remove/clear: PUT/DELETE /cart routes keep items and totals in sync with the backend.

13.5 Checkout & Payment Cycle

- CheckoutPage loads addresses, payment providers, and calls POST /shipping/calculate for the estimate.
- Placing the order: orderApi.create() → POST /orders → OrderController@store validates, loads the cart, applies delivery cost (default 5000 TSh when delivery is required) and the optional winga fee (subtotal × rate%), updates the cart row to status pending with an ORD- number, returns the fresh order.
- Payment: paymentApi.initiate() → POST /payments/initiate → PaymentController@initiate creates a pending payment. ClickPesa and cash are auto-confirmed: payment completed, order.markAsPaid() (status paid + inventory decremented). Other mobile-money providers await an employee confirmation or the webhook.
- Employee confirmation: OrderManagementPage → orderManageApi.updateStatus(order, "paid") → OrderController::confirmPaidOrder runs the full auto-journal chain (see §13.6).

13.6 Order Lifecycle & Auto-Journaling

OrderController@updateStatus is the heart of the system. Inside a database transaction with a row lock it transitions statuses and triggers accounting. Only the paid and cancelled transitions post journals; shipping transitions are plain status updates.

```
pending_payment (cart) ──checkout──▶ pending
pending ──confirm──▶ paid ← confirmPaidOrder:
1. decrement each variant inventory
2. write sale InventoryTransactions
3. compute COGS (Σ qty × cost_price; throws if missing)
4. postSale journal: DR Cash 1020 / CR Sales 4010
(VAT Output 2500 + Shipping 4020 + winga accrual 5110/2100)
CR Inventory 1200 / DR COGS 5010
5. createCommission (employee, profit × rate)
6. createWingaCommission (fee − TRA TDS 5%, net payable)
paid ──▶ processing ──▶ shipped ──▶ delivered (plain updates + tracking)
paid/processing ──cancel──▶ cancelled ← reversePaidOrder:
restock inventory + return transactions + reverseSale +
reverseCommissions (clawback paid, delete pending) +
reverseWingaCommissions
any item return ──▶ returnItems:
restock, write return transactions, postReturn partial journal,
```

```
adjustCommissionForReturn + adjustWingaCommissionForReturn;
```

```
fully returned orders auto-cancel
```

13.7 Employee Cycle

- Employee login → EmployeeDashboard (stats + branch-scoped accounting issues).
- Order desk: OrderManagementPage → orderManageApi → updateStatus, updateDelivery (tracking), returnItems.
- Customers: CustomerManagementPage → customerApi (toggle status, delete).
- Support: SupportInboxPage → supportApi.reply/updateStatus.
- Earnings: EmployeeEarningsPage → commissionApi.getMyEarnings → GET /commissions/my-earnings.
- Inbox: EmployeeInboxPage → conversationApi (owner_employee threads).

13.8 Owner Cycle — HR, Inventory, Procurement

- Employees: EmployeeManagementPage → employeeApi.create sends multipart form-data (guarantors + documents). Default password = strtoupper(name); employee_code EMP-xxxxxx.
- Inventory: InventoryManagementPage → inventoryApi.adjust → InventoryController@adjust posts the adjustment journal (1200/5100/3010) and triggers the low-stock scan.
- Purchase orders: PurchaseOrderPage → purchaseOrderApi → PurchaseOrderController. Receiving (POST /purchase-orders/{id}/receive) increments stock, writes purchase transactions, and posts DR Inventory 1200 / CR Cash 1020.
- Suppliers: SupplierManagementPage → supplierApi CRUD + documents.
- Stock alerts: StockAlertsPage → stockAlertApi acknowledge/resolve.

13.9 Accounting Cycle

- Chart of accounts: ChartOfAccountsPage → accountApi; system accounts (is_system) are protected from edit/delete.
- Manual entries: JournalEntryCreatePage → journalApi.create (draft, must balance), then post or void from the detail page.
- Live statements: TrialBalance/ProfitLoss/BalanceSheet/GeneralLedger pages → accountingReportApi; general-ledger requires an account_id.
- Generated reports: ReportsPage → generate-monthly/yearly; AiSuggestions persist to the report.
- Accounting issues: AccountingDashboardPage → accountingIssuesApi.get → AccountingIssuesController bucket scan (unconfirmed payments, pending deliveries, missing cost prices, drafts, voids, pending commissions, unbalanced trial balance, low stock).

13.10 Winga Commission Cycle

- Owner registers a winga (name, TIN/NIDA, rate) via WingaManagementPage.
- At checkout the customer's order stores winga_id + winga_fee.

• On payment confirmation, createWingaCommission() accrues gross/fee, withholding tax (5%), net.

• Owner pays (WingaCommissionPage → wingaCommissionApi.pay/payAll) → payout journal: DR Winga Payable 2100 / CR Cash 1020 (net) / CR WHT Payable 2120 (TDS).

13.11 Supplier Portal Cycle

• Supplier login → SupplierPortalPage → supplierPortalApi.getProfile() matches the user email to a Supplier record.

• GET /supplier-portal/purchase-orders lists the supplier's POs; supplierUpdateStatus moves draft → ordered → received (received also stamps received_date).

13.12 Superadmin Onboarding Cycle

• SuperadminDashboard → superadminApi.getStats() → GET /superadmin/stats.

• Create owner → POST /superadmin/owners → default password returned; OwnerProfile created (trial, starter, 50 products, 5 employees) and a Business row seeded.

• OwnerDetailPage exposes subscription/limits/password management; BrandingPage → PUT branding + POST branding-logo.

13.13 Messaging & Read-Receipts Cycle

• Threads: conversationApi.index / create → GET|POST /conversations; inbox pages list open threads (sidebar) with unread dots (hasUnread = any incoming message with is_read = false).

• Send: inbox page → conversationApi.sendMessage → POST /conversations/{id}/messages → ConversationController@sendMessage stores the row (is_read = false) and returns it; the sender renders a grey double tick.

• Read: recipient opens the thread → GET /conversations/{id} → show() validates the role/tenant then bulk-updates incoming messages (sender_id != user) to is_read = true before returning the conversation.

• Receipt delivery: every inbox page polls GET /conversations and GET /conversations/{id} every 15 s; the sender's grey tick turns blue once the recipient has viewed the message. No websockets are used.

14. Documentation & Diagrams

PDF Documents (docs/)

File	Audience	Notes
User_Manual_EN.pdf	End users	English user manual, 24 chapters
User_Manual_SW.pdf	End users	Swahili user manual, 24 chapters
Supplier_Manual.pdf	Suppliers	Supplier portal manual
Developer_Documentation.pdf	Developers & admins	This document (v2.1)

System Diagrams (docs/)

Diagram	Contents
ERD.drawio (+ PDF)	All tables and relationships, crow's foot notation
ClassDiagram.drawio (+ PDF)	Model classes with attributes, methods, and relationships
UseCase.drawio (+ PDF)	Actors and use cases across all roles
SequenceDiagrams.drawio (+ PDF)	7 flows: customer checkout, order processing, branch + employee management, accounting, commission, inventory, purchase orders

Generator Scripts

- `generate_manual.py` — user manuals (EN + SW) using ReportLab.
- `generate_supplier_manual.py` — supplier portal manual.
- `generate_dev_doc.py` — this developer documentation.
- `generate_diagrams.py` — drawio diagram generator.

15. Development Commands & Environment

15.1 Local Development vs Production

The same repositories serve both environments. Local development uses the Vite dev server against the local Laravel API; deployment builds the frontend with the production API URL. The two never interfere because the frontend URL is a build-time environment variable.

Concern	Local development	Production
Frontend URL	`http://localhost:5173`	`https://electroshophub.online`
API base (VITE_API_URL)	`http://localhost:8000/api`	`https://api.electroshophub.online/api`
Backend host	php artisan serve (localhost:8000)	Laravel Cloud (api.electroshophub.online)
Database	SQLite (database/database.sqlite)	Provisioned database on Laravel Cloud
File storage	local disk	S3 (FILESYSTEM_DISK=s3)
Environment file	.env (untracked, git-ignored)	Environment variables on Laravel Cloud

15.2 Frontend (erp-electronics/)

Command	Purpose
npm install	Install dependencies (Node ^22.18 or >= 24.12)
npm run dev	Start Vite dev server (port 5173)
npm run build	Production build
npm run lint	Run oxlint + eslint
npm run format	Format with oxfmt
npm run test:unit	Run Vitest unit tests
npm run test:e2e	Run Playwright end-to-end tests

15.3 Backend (erp-electronics-api/)

Command	Purpose
composer install	Install PHP dependencies
php artisan serve	Start dev server (port 8000)
php artisan migrate	Run migrations
php artisan migrate:fresh --seed	Reset database and re-seed

Command	Purpose
php artisan report:daily	Generate daily report
php artisan orders:cleanup-unpaid	Clean up unpaid orders
php artisan superadmin:reset-password	Reset superadmin password if 6 months elapsed
php artisan accounting:generate-reports --with-suggestions	Generate monthly reports with AI suggestions
php artisan accounting:close-year	Post year-end closing entries
php artisan schedule:work	Run the scheduler
php artisan route:list	List routes
php artisan test	Run the test suite

15.4 Environment Variables

```
# Frontend (.env — untracked, local only)
VITE_API_URL=http://localhost:8000/api

# Production build sets instead:
VITE_API_URL=https://api.electroshophub.online/api

# Backend (.env)
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database.sqlite
GEMINI_API_KEY=your-gemini-api-key
FILESYSTEM_DISK=local # "s3" on Laravel Cloud
AWS_ACCESS_KEY_ID= # when using S3
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false
```

15.5 Documentation Regeneration

- python3 generate_dev_doc.py → docs/Developer_Documentation.pdf
- python3 generate_manual.py → docs/User_Manual_EN.pdf + User_Manual_SW.pdf
- python3 generate_supplier_manual.py → docs/Supplier_Manual.pdf
- python3 generate_diagrams.py → drawio files (open + export to PDF in diagrams.net)

16. Deployment

16.1 Production Endpoints

Service	URL	How served
Frontend	`https://electroshophub.online`	Static SPA build (dist/) with SPA redirects
Backend API	`https://api.electroshophub.online/api`	Laravel Cloud, custom domain api.electroshophub.online
Health check	`https://api.electroshophub.online/up`	GET /up returns 200 when the app is healthy

16.2 Backend — Laravel Cloud

- Pushing to the main branch triggers an automatic deployment.
- Custom domain api.electroshophub.online is mapped to the production environment (environment id env-a2578b2b-6a5c-4d19-9c2e-d927b4edafde; app id app-a2578b29-e3ec-4754-a717-a6a208f4f512).
- File storage uses S3 (FILESYSTEM_DISK=s3) with the AWS_* variables set on Laravel Cloud.
- Deployment verification: `cloud command:run <env-id> --cmd="php artisan test" -n`.

16.3 Frontend — Production Build

- Build with the production API URL: set `VITE_API_URL=https://api.electroshophub.online/api` at build time, then run `npm run build`.
- Publish the `dist/` directory to the frontend host serving electroshophub.online with SPA redirects to index.html.
- The API accepts the frontend origin via CORS (allowed_origins: *).

16.4 Keeping Local Development Running

- The local `.env` keeps `VITE_API_URL=http://localhost:8000/api`; it is untracked so production builds never pick it up.
- Run the backend with `php artisan serve` (or `php artisan schedule:work` for scheduled tasks) and the frontend with `npm run dev`.
- Local SQLite means migrations can be reset freely (`php artisan migrate:fresh --seed`) without touching production data.
- Production is only affected by a deploy — nothing local touches the live environment.

Rate-limit keys are IP-based, so an entire office sharing one public IP shares the 5/min login budget. This is intentional to block brute-force traffic.

17. Implementation Notes & Known Gotchas

#	Note
1	The branch routes are documented "owner only" but are not wrapped in the owner middleware — any authenticated user can call them. UI gates them by role.
2	branch_id on orders/employees and winga_id on orders are plain unsigned columns with no foreign key constraints.
3	journal_entries.source_type is a foreignId column without an actual FK constraint.
4	winga_wht_rate is seeded in two migrations (2026_07_31_000004 and 2026_08_01_000007); the first migration's down() does not remove it.
5	Order status enum: pending, pending_payment, inactive, paid, processing, shipped, delivered, cancelled (default pending_payment).
6	Tenant resolution depends on the X-Business-Id header with fallback to the first owned business; ownerId falls back to the owner's own user id for legacy installs.
7	OwnerProfile productCount()/employeeCount() are global counts, not tenant-scoped — treated as approximate.
8	The payments webhook currently has no signature validation (documented TODO).
9	PaymentController auto-confirmation path (ClickPesa/cash) marks orders paid without posting a journal entry — only the OrderController::updateStatus path posts journals.
10	OrderItem.returned_quantity is used directly in OrderController::returnItems but is not in the model's fillable list (updated via update()).
11	Sanctum tokens have no expiration by default; sessions are protected by the frontend idle timeout and server-side token revocation on logout.